PROJETO INTEGRADOR

Sistemas Operacionais

Simulador interativo, demonstração no xv6 e o tema em um SO atual

Objetivo: Mostrar que o grupo domina um tema de SO de três formas: construindo um simulador, demonstrando o algoritmo funcionando no xv6 e explicando como um sistema operacional atual resolve o mesmo problema.

Formato	6 grupos de 8 estudantes, um tema por grupo
Valor	2,5 pontos na média
Entrega	Até 25/11, tudo publicado no Padlet do grupo
Apresentações	26 e 27/11 · até 40 minutos por grupo
Avaliação dos colegas	Até 29/11
Materiais de apoio	Livro do xv6 (fornecido pelo professor) e Guia do Padlet

O projeto em uma página

Atividade	O que o grupo faz
1. Simulador	Cria com IA um simulador ou jogo em HTML sobre o tema.
2. Demonstração no xv6	Mostra no código do xv6 como o tema funciona, implementa o que for preciso e faz um experimento ao vivo
3. O tema em um SO atual	Explica como um SO atual (Linux, Windows, Android, macOS…) trata o tema.
4. Padlet	Tudo deve estar disponibilizado em um Padlet.

Os 6 temas

Cada grupo recebe um tema. O capítulo indicado é do livro do xv6, a referência principal para a Atividade 2.

Grupo	Tema	Livro do xv6	Onde está no código
1	Processos e threads	Cap. 1, 2 e 7	kernel/proc.c, kernel/swtch.S
2	Escalonamento	Cap. 7	scheduler() em kernel/proc.c

Grupo	Tema	Livro do xv6	Onde está no código
3	Sincronização, comunicação e deadlock	Cap. 1 e 6	kernel/spinlock.c, kernel/pipe.c
4	Memória	Cap. 3	kernel/vm.c, kernel/kalloc.c
5	Sistema de arquivos	Cap. 8	kernel/fs.c, kernel/log.c
6	E/S e hardware	Cap. 5	kernel/trap.c, uart.c, virtio_disk.c

ATIVIDADE 1 /3 Simulador ou jogo em HTML

O que entregar

- Um simulador ou jogo em um arquivo HTML que roda no navegador, criado com ajuda de IA.
- Uma justificativa técnica explicando as escolhas técnicas da(o) simulação/jogo (utilizar o próprio espaço do padlet para isso).
- As duas coisas publicadas no Padlet do grupo (veja o Guia do Padlet).

Passo a passo

1. Escolham o que o simulador vai mostrar (veja as ideias abaixo).
2. Gerem utilizando IA.
3. Revisem a interface usando as 10 heurísticas de Nielsen (https://www.nngroup.com/articles/ten-usability-heuristics/).
4. Publiquem o simulador e escrevam a justificativa técnica no Padlet.

Ideias de simulador por tema

Tema	Ideias (escolham uma ou proponham outra)
Processos e threads	Jogo de gerenciar processos: criar, bloquear, acordar e encerrar, vendo os estados mudarem Processos × threads: o que é compartilhado e o que é separado
Escalonamento	FCFS, SJF, Round-Robin e prioridade com diagrama de Gantt e tempos médios Jogo: o jogador escolhe quem roda e compara seu resultado com o algoritmo
Sincronização, comunicação e deadlock	Contador compartilhado com e sem semáforo Produtor-consumidor animado Jantar dos filósofos ou grafo de alocação de recursos com detecção de ciclo
Memória	Tradução de endereço virtual para físico com tabela de páginas Substituição de páginas (FIFO, LRU, Ótimo) contando as faltas
Sistema de arquivos	Disco em blocos com alocação contígua, encadeada e indexada Inode com ponteiros diretos e indiretos crescendo junto com o arquivo
E/S e hardware	Escalonamento de disco (FCFS, SSTF, SCAN) com o braço se movendo Caminho de uma interrupção: dispositivo, controlador, CPU e tratador Polling × interrupção × DMA

Prompt sugerido para começar

Troquem o texto entre colchetes. Depois, peçam melhorias aos poucos, uma de cada vez.

```
Você é desenvolvedor front-end e professor de Sistemas Operacionais.
Crie um simulador interativo em UM ÚNICO arquivo HTML (HTML, CSS e
JavaScript, sem bibliotecas externas) sobre: [TEMA / ALGORITMO].
Requisitos:
- O usuário define a entrada (ex.: processos e tempos, páginas).
- A simulação roda passo a passo: botões Próximo, Pausar, Reiniciar.
- O estado atual fica em destaque e cada passo é explicado em uma
  frase curta.
- Siga as 10 heurísticas de usabilidade de Nielsen.
- Comente o código explicando a lógica do algoritmo.
Ao final, liste onde cada heurística foi aplicada.
```

Referência: Nielsen Norman Group, "10 Usability Heuristics for User Interface Design" — nngroup.com/articles/ten-usability-heuristics

Justificativa técnica

1. O que o simulador mostra: qual conceito de SO e qual algoritmo.
2. Como funciona: a lógica do algoritmo e as estruturas de dados usadas, explicadas pelo grupo.
3. Decisões tomadas e por quê (ex.: passo a passo em vez de tempo real).
4. Simplificações: o que é diferente ou igual a de um SO real.

> 💡 **Publicação do simulador**
> *Coloquem o simulador em um endereço público (ex.: GitHub Pages) ou anexem o arquivo .html no Padlet para download. O Guia do Padlet mostra os dois caminhos.*

ATIVIDADE 2 /3 Demonstração no xv6

Objetivo

Mostrar em aula, no código e funcionando, como o xv6 trata o tema do grupo. A turma deve sair entendendo o algoritmo, não só vendo a saída na tela.

Passo a passo

1. Leiam o capítulo do livro do xv6 indicado para o tema.
2. Escolham uma ideia de demonstração da lista do tema (ou proponham outra ao professor).
3. Localizem e entendam o código do núcleo envolvido.
4. Implementem o que for necessário: um programa de usuário, uma syscall nova ou uma mudança no núcleo..
5. **Publiquem no Padlet um vídeo contendo a demonstração do código criado ou alterado, demonstrando o funcionamento.**

> 💡 **Lembretes do ambiente**
> - Programa novo em user/ precisa ser registrado na lista UPROGS do Makefile.
> - Syscall nova segue os mesmos passos do Lab 02 (user.h, usys.pl, syscall.h, syscall.c, sysproc.c).
> - make qemu roda com 3 CPUs; make CPUS=1 qemu roda com 1 CPU.
> - Ctrl-P mostra a lista de processos e seus estados; Ctrl-A depois X encerra o QEMU.

Ideias de demonstração por tema

Grupo 1 · Processos e threads

- Ciclo de vida: criar filhos que terminam antes do pai chamar wait() e mostrar os zumbis com Ctrl-P.
- Criar uma syscall que conta quantos processos estão em cada estado.
- Explicar com o gdb a troca de contexto entre threads do núcleo (swtch.S e sched()).

Grupo 2 · Escalonamento

- Medir como a CPU é dividida entre processos com 1 e com 3 CPUs (partindo do exemplo resolvido abaixo).
- Contar quantas vezes cada processo foi escalonado e mostrar com uma syscall ou com Ctrl-P.
- Implementar uma prioridade simples e comparar com o round-robin original.

Grupo 3 · Sincronização, comunicação e deadlock

- Provocar uma condição de corrida entre processos que somam em um arquivo e corrigir com um semáforo feito de pipe.
- Provocar um deadlock com dois pipes (os dois processos esperam ler) e corrigir mudando a ordem das operações.
- Explicar por que acquire() desliga as interrupções (push_off) e o que aconteceria sem isso.
- Atenção: a demonstração precisa cobrir pelo menos dois dos três assuntos do tema.

Grupo 4 · Memória

- Medir o tempo do fork() com heap pequeno e com heap grande e explicar por que existe o copy-on-write.
- Comparar as tabelas de páginas de dois processos (indo além do vmprint do Lab 03).
- Mostrar o que acontece ao acessar um endereço fora do heap (falta de página) e onde o núcleo trata isso.

Grupo 5 · Sistema de arquivos

- Escrever um arquivo até o limite do inode e explicar o número obtido (blocos diretos + indiretos).
- Mostrar o inode e os blocos de um arquivo, seguindo bmap() em fs.c.
- Explicar como o log (log.c) protege o sistema de arquivos contra quedas no meio de uma escrita.

Grupo 6 · E/S e hardware

- Contar as interrupções por dispositivo (timer, teclado, disco) e mostrar com uma syscall nova.
- Seguir uma tecla com o gdb: da interrupção do UART até o shell (uart.c, console.c).
- Medir quantas interrupções de disco uma escrita em arquivo gera e explicar o papel do virtio_disk.c.

Exemplo resolvido: como estruturar a demonstração

O exemplo abaixo mostra o formato esperado (previsão, execução, explicação). Ele é só um modelo: o grupo de Escalonamento pode partir dele, mas precisa ir além.

`user/escalona.c` — vários processos disputando a CPU durante o mesmo intervalo:

```
// escalona.c - quantas "voltas" cada processo consegue dar
// Uso: escalona [n] [ticks]   (padrao: 3 processos, 100 ticks)
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int n = 3, dur = 100;
  int fd[2];
  if(argc > 1) n = atoi(argv[1]);
```

```
  if(argc > 2) dur = atoi(argv[2]);

  pipe(fd);                    // filhos mandam o resultado ao pai
  int inicio = uptime() + 5;   // todos comecam juntos
  for(int i = 0; i < n; i++){
    if(fork() == 0){
      while(uptime() < inicio)
        ;
      int conta = 0;
      while(uptime() < inicio + dur){
        for(volatile int k = 0; k < 10000; k++)
          ;                    // trabalho puro de CPU
        conta++;               // 1 bloco = 10 mil voltas
      }
      int msg[2] = { i, conta };
      write(fd[1], msg, sizeof(msg));
      exit(0);
    }
  }
  for(int i = 0; i < n; i++){
    int msg[2];
    read(fd[0], msg, sizeof(msg));
    printf("processo %d: %d blocos\n", msg[0], msg[1]);
  }
  for(int i = 0; i < n; i++)
    wait(0);
  printf("fim: %d processos durante %d ticks\n", n, dur);
  exit(0);
}
```

Registrar no Makefile (dentro de UPROGS) e rodar:

```
$U/_escalona\

$ make CPUS=1 qemu          # depois: make qemu (3 CPUs)
$ escalona 1                # 1 processo sozinho
$ escalona                  # 3 processos
$ escalona 6                # 6 processos
```

Resultados obtidos em um computador de teste (os números mudam de máquina para máquina; o que importa é a proporção):

Execução	1 CPU	3 CPUs (máquina com 2 núcleos)
escalona 1	≈ 269 mil blocos	≈ 248 mil blocos
escalona (3 processos)	≈ 82 mil cada	≈ 163 mil cada
escalona 6	≈ 42 mil cada	≈ 83 mil cada

ATIVIDADE 3 /3

O tema em um SO atual

O que apresentar

1. Escolham um SO atual: Linux, Windows, Android, macOS, iOS ou outro.
2. Expliquem como ele trata o tema do grupo.
3. Mostrem as fontes usadas no último slide (documentação oficial, livros, artigos).

Sugestões de assunto

Tema	Sugestões
Processos e threads	Linux: clone() e threads como tarefas Android: o processo Zygote Windows: processos, threads e fibers
Escalonamento	Linux: do CFS ao EEVDF Windows: escalonamento por prioridades Celulares: núcleos grandes e pequenos (big.LITTLE)
Sincronização, comunicação e deadlock	Linux: futex e o detector lockdep Android: comunicação pelo Binder Bancos de dados: detecção de deadlock
Memória	Linux: copy-on-write no fork e o OOM killer Android: o Low Memory Killer Windows e macOS: compressão de memória
Sistema de arquivos	ext4: extents e journaling APFS: copy-on-write e snapshots NTFS ou Btrfs
E/S e hardware	Linux: io_uring SSDs NVMe e múltiplas filas Drivers e plug-and-play (USB)

Apresentação (até 40 min)

Momento	Conteúdo
Simulador	Demonstração ao vivo e principais decisões da justificativa
xv6	Código, execução e explicação
SO atual	Como o SO escolhido trata o tema e a comparação com o xv6

O que publicar no Padlet

Cada grupo cria um Padlet seguindo o Guia do Padlet. Até 25/11, ele deve ter:

Coluna do Padlet	Conteúdo
1. Simulador	Link público do simulador ou o arquivo .html para download, e o PDF da justificativa técnica
2. Demonstração no xv6	Vídeo demonstrando o código e a aplicação rodando no xv6.
3. SO atual	Slides da apresentação (PDF)
4. Avaliações	Um post "Avalie nosso trabalho aqui", onde os colegas comentam

Avaliação: 2,5 pontos

Parte	Quem avalia	Pontos
Trabalho do grupo	Professor	1,5
Qualidade das avaliações que você fez dos outros grupos	Professor	0,5
Nota que os colegas deram ao trabalho do grupo	Colegas	0,5
Total		2,5

Trabalho do grupo (1,5 · professor)

Atividade	O que é avaliado	Pontos
1. Simulador	Funciona e está correto; aplica as heurísticas de Nielsen; justificativa técnica clara e completa	0,5
2. Demonstração no xv6	Código correto; explicação clara do algoritmo; experimento com previsão e explicação	0,5
3. SO atual	Explicação correta; comparação com o xv6; fontes confiáveis	0,5

Avaliação por pares (0,5 · professor)

Você avalia um outro grupo, até 29/11, comentando no post "Avaliações" do Padlet de cada grupo. Cada comentário tem uma descrição seguindo as diretrizes de avaliação propostas. Segue o que deverá ser feito:

- Comentar no Padlet do grupo e deixar uma critica de acordo com as rubricas esperadas (apenas o comentário, não é necessário dar uma nota nesse post);
- Acesse o formulário de avaliação, e nele você deverá colocar o mesmo comentário e a nota dada ao trabalho para o grupo avaliado.
- A sequência de avaliação será a seguinte: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 1. Ou seja, os integrantes do grupo 1 avaliam os do grupo e idem.

> 💡 **Rúbrica avaliativa: para dar a nota, considere**
>
> - O conteúdo apresentado está tecnicamente correto?
> - A explicação foi clara?
> - O simulador/jogo ajuda a entender o conceito e é fácil de usar?
> - A demonstração no xv6 mostrou o algoritmo funcionando? A demonstração foi clara?
> - O Padlet está bem organizado e claro?
> - O Projeto completo facilita entender o conteúdo abordado?

Como o professor avalia o seu comentário	Pontos
Descrição específica e coerente com a nota (cita o que viu, não só "gostei")	0,4
Comentário construtivo, com pelo menos um ponto a melhorar	0,1

Nota dos colegas (0,5)

O grupo recebe a média das notas (0 a 5) dadas pelos colegas, convertida para 0,5 ponto:

```
nota dos colegas = média recebida ÷ 5 × 0,5
exemplo: média 4,2  →  4,2 ÷ 5 × 0,5 = 0,42
```

Organização do grupo

Sugestão de papéis para 8 pessoas. Todos revisam tudo antes da entrega. Recomenda-se a escolha de 1 pessoa para ser a líder do grupo e ficar responsável para dialogar com o professor e com a equipe.

Papel	Responsabilidade
Coordenação	Cronograma, Padlet e contato com o professor
Simulador	Construção, testes e justificativa técnica
xv6	Código, experimento e roteiro da demonstração
SO atual	Pesquisa, slides e comparação com o xv6

Cronograma

Data	O que acontece
até 05/11	Grupos formados e temas definidos
20/11	Padlet criado, ideia do xv6 escolhida e primeira versão do simulador
até 25/11	Entrega: tudo publicado no Padlet
26 e 27/11	Apresentações (3 grupos por dia)
até 29/11	Avaliação dos colegas no Padlet

Uso de IA

- A IA é permitida e incentivada para criar o simulador.
- O grupo responde por tudo o que entrega: confiram os resultados com exemplos feitos à mão.
- Registrem na justificativa técnica os principais prompts usados.
- No xv6 e na apresentação, usem a IA como tutor, não para gerar respostas prontas.

> 🤖 **INSTRUÇÃO AO TUTOR DE IA**
>
> *Cole esta instrução no início da conversa com a IA ao estudar o código do xv6:*
>
> *"Você é tutor de Sistemas Operacionais e estou estudando o xv6 (versão RISC-V, xv6-labs-2024). Não me dê código pronto nem a resposta final. Faça perguntas que me levem a entender o código, dê pistas aos poucos e peça que eu explique com minhas palavras. Se não tiver certeza sobre algum detalhe desta versão do xv6, diga isso e indique onde conferir no código."*